

Overall Project Documentation

University of Hyderabad — Placement Management System (PMS)

Table of Contents

1. Project Overview
 2. Tech Stack
 3. System Architecture
 4. Folder Structure
 5. Environment Setup
 6. Running the Application
 7. API Reference
 8. Frontend Pages & Routes
 9. Role-Based Access Control
 10. Database Schema
 11. Design System
 12. Testing
 13. CI/CD
 14. Sprint Roadmap
-

1. Project Overview

The **Placement Management System (PMS)** is a full-stack web application built for the **University of Hyderabad (UoH)** — School of Computer and Information Sciences. It streamlines and digitalises the campus placement process by connecting students, faculty (TPO), companies, and administrators on a single unified platform.

Field	Details
University	University of Hyderabad
Department	School of Computer and Information Sciences (SCIS)
Team	22MCCE13 · 22MCCE16
Methodology	Agile Scrum (2-week sprints)
Project Year	2025–2026

Key Features (Sprint 1 — Implemented)

- User registration with role selection (Student / Faculty / Company)
- Firebase Authentication (email+password and Google SSO)

- Email verification flow
- Forgot & Reset password
- Role-Based Access Control (RBAC) — Student, Faculty, Company, Admin
- Admin panel — view all users, change roles, activate/deactivate/delete accounts
- Responsive UI with split-panel auth pages and sidebar navigation

2. Tech Stack

Backend

Layer	Technology
Runtime	Node.js v20 LTS
Framework	Express.js v4
Authentication	Firebase Admin SDK v12
Database	Cloud Firestore (Firebase)
Validation	express-validator
Security	helmet, cors, express-rate-limit, morgan
Testing	Jest 29 + Supertest
Environment	dotenv

Frontend

Layer	Technology
Framework	React 18
Build Tool	Vite 5
Styling	Tailwind CSS 3.4
Routing	React Router DOM v6
Auth SDK	Firebase Client SDK v10
Notifications	react-hot-toast
Icons	react-icons v5

DevOps

Tool	Purpose
Docker + docker-compose	Containerisation
nginx (alpine)	Frontend static file serving + API proxy
GitHub Actions	CI/CD pipeline

3. System Architecture

```

                                Browser (React SPA)
LoginPage · RegisterPage · Dashboard · AdminUsersPage · ...

                                HTTP (VITE_API_URL / nginx proxy)

                                Express.js REST API
/api/register /api/login /api/me
/api/admin/users /api/admin/users/:id/role ...

Middleware: authenticate → authorize → validate → controller


Firebase Auth                  Cloud Firestore
(token verify,                 Collection: users
password reset,                Fields: uid, name,
email verify)                  email, role, status,
                                department, createdAt
```

4. Folder Structure

```
src/
  backend/
    src/
      app.js                # Express app setup (middleware, routes)
      server.js              # HTTP server entry point
      config/
        constants.js         # ROLES, ACCOUNT_STATUS, UNIVERSITY_ROLES
        firebase.js          # Firebase Admin SDK init
      controllers/
        authController.js    # register, login, verifyEmail, forgotPassword, resetPassword
        adminController.js   # listUsers, updateUserRole, updateUserStatus, deleteUser
      middleware/
        auth.js              # authenticate() - verifies Firebase ID token
        rbac.js              # authorize(role) - enforces role check
        validators.js        # express-validator rule sets
        errorHandler.js      # Global error handler
      routes/
        authRoutes.js        # /api/* public + protected auth routes
```

```

        adminRoutes.js      # /api/admin/* admin-only routes
scripts/
  seedAdmin.js             # One-time admin account seeder
utils/
  validationHelper.js
tests/                     # Jest unit tests
.env                       # Backend secrets (not committed)
package.json
Dockerfile

frontend/
  src/
    App.jsx                # Router + protected route wrappers
    main.jsx               # React DOM entry
    index.css              # Full design system (Tailwind @layer)
    assets/
      Gemini_Generated_Image_lkoyazlkoyazlkoy.png # UoH logo
    components/
      Layout.jsx           # Sidebar + Navbar shell
      Sidebar.jsx          # Dark sidebar with role-aware nav links
      Navbar.jsx           # Top bar with avatar, bell, logout
    config/
      firebase.js          # Firebase Client SDK init
    contexts/
      AuthContext.jsx      # Auth state, register, login, logout helpers
    pages/
      LoginPage.jsx
      RegisterPage.jsx
      ForgotPasswordPage.jsx
      ResetPasswordPage.jsx
      VerifyEmailPage.jsx
      DashboardPage.jsx
      AdminUsersPage.jsx
      NotFoundPage.jsx
      UnauthorizedPage.jsx
    services/
      api.js               # Axios wrappers (authAPI, adminAPI)
  tailwind.config.js      # Design tokens (colors, shadows, animations)
  .env                    # VITE_API_URL, VITE_FIREBASE_* (not committed)
  package.json
  Dockerfile

docker-compose.yml
Sprint1_Documentation.md

```

5. Environment Setup

Backend .env

```
PORT=5005
NODE_ENV=development
FRONTEND_URL=http://localhost:5173
UNIVERSITY_EMAIL_DOMAIN=uohyd.ac.in

# Firebase Admin SDK
FIREBASE_PROJECT_ID=your-project-id
FIREBASE_CLIENT_EMAIL=firebase-adminsdk-xxx@your-project.iam.gserviceaccount.com
FIREBASE_PRIVATE_KEY="-----BEGIN PRIVATE KEY-----\nXXXXXX\n-----END PRIVATE KEY-----\n"
```

Frontend .env

```
VITE_API_URL=http://localhost:5005/api
VITE_UNIVERSITY_EMAIL_DOMAIN=uohyd.ac.in

# Firebase Client SDK
VITE_FIREBASE_API_KEY=AIzaSy...
VITE_FIREBASE_AUTH_DOMAIN=your-project.firebaseio.com
VITE_FIREBASE_PROJECT_ID=your-project-id
VITE_FIREBASE_STORAGE_BUCKET=your-project.appspot.com
VITE_FIREBASE_MESSAGING_SENDER_ID=123456789
VITE_FIREBASE_APP_ID=1:123456789:web:abc123
```

6. Running the Application

Local Development (Recommended)

Terminal 1 — Backend:

```
cd src/backend
npm install           # first time only
node src/server.js   # runs on http://localhost:5005
```

Terminal 2 — Frontend:

```
cd src/frontend
npm install           # first time only
npm run dev           # runs on http://localhost:5173
```

Open **http://localhost:5173** in your browser.

Docker

```
cd src
docker compose up --build
```

- Frontend: **http://localhost:3001**
- Backend: **http://localhost:5001**

Note: Dockerfiles use Node v20 LTS (compatible). Local machine must have Docker Desktop running.

7. API Reference

Base URL

- Local: **http://localhost:5005/api**
- Docker: **http://localhost:5001/api**

Authentication

All protected endpoints require a Firebase ID token in the **Authorization** header:

Authorization: Bearer <firebase-id-token>

7.1 Auth Endpoints

POST /api/register Register a new user account.

Request Body:

```
{
  "name": "Krishna Vaibhav",
  "email": "22mcce13@uohyd.ac.in",
  "password": "Secure@123",
  "department": "Computer Science",
  "role": "Student"
}
```

Response 201:

```
{
  "message": "Registration successful. Please verify your email.",
  "uid": "firebase-uid"
}
```

Notes: - Students must use @uohyd.ac.in email - New accounts are created with status **Inactive** - Password is stored in Firebase Auth (not Firestore)

POST /api/login Login and get user profile from Firestore.

Request Body:

```
{
  "email": "22mcce13@uohyd.ac.in",
  "password": "Secure@123"
}
```

Response 200:

```
{
  "message": "Login successful.",
  "user": {
    "uid": "...",
    "name": "Krishna Vaibhav",
    "email": "22mcce13@uohyd.ac.in",
    "role": "Student",
    "status": "Active",
    "department": "Computer Science"
  }
}
```

GET /api/me Get the authenticated user's profile.

Headers: Authorization: Bearer <token>

Response 200:

```
{
  "user": { "uid": "...", "name": "...", "email": "...", "role": "...", "status": "..." }
}
```

POST /api/forgot-password Send a password reset email.

Request Body: { "email": "user@example.com" }

Response 200: { "message": "Password reset email sent." }

POST /api/reset-password Reset password using Firebase oobCode (handled client-side via Firebase SDK).

7.2 Admin Endpoints

All require Authorization: Bearer <admin-token>

GET /api/admin/users List all users with optional filters.

Query Params: ?role=Student&status=Inactive

Response 200:

```
{
  "users": [
    { "uid": "...", "name": "...", "email": "...", "role": "Student", "status": "Inactive",
  },
  "total": 1
}
```

PATCH /api/admin/users/:id/role Update a user's role.

Request Body: { "role": "Admin" }

Response 200: { "message": "Role updated successfully." }

PATCH /api/admin/users/:id/status Activate or deactivate a user account.

Request Body: { "status": "Active" } (*Active / Inactive / Deactivated*)

Response 200: { "message": "Status updated successfully." }

DELETE /api/admin/users/:id Permanently delete a user from Firestore and Firebase Auth.

Response 200: { "message": "User deleted successfully." }

8. Frontend Pages & Routes

Route	Component	Access
/login	LoginPage	Public
/register	RegisterPage	Public
/forgot-password	ForgotPasswordPage	Public
/reset-password	ResetPasswordPage	Public (via email link)

Route	Component	Access
/verify-email	VerifyEmailPage	Authenticated (unverified)
/dashboard	DashboardPage	Authenticated + Active
/admin/users	AdminUsersPage	Admin only
/unauthorized	UnauthorizedPage	Public
*	NotFoundPage	Public

9. Role-Based Access Control

Role	Description	Email Requirement	Dashboard Access
Student	UoH enrolled student	Must use @uohyd.ac.in	Student dashboard
Faculty	TPO / Placement officer	Any email	Faculty dashboard
Company	Recruiter / HR	Any email	Company dashboard
Admin	System administrator	Any email	Full access + Admin panel

RBAC Flow:

```
Request → authenticate() middleware
        → verifies Firebase ID token
        → attaches req.user (uid, role)
        → authorize('Admin') middleware
        → checks req.user.role === 'Admin'
        → 403 if mismatch, else next()
```

Account Status Flow:

```
Register → status: Inactive
        → email verified → status: Active (set by admin or auto)
        → Admin activates → status: Active
        → Admin deactivates → status: Deactivated
```

10. Database Schema

Firestore Collection: users

Field	Type	Description
<code>uid</code>	string	Firebase Auth UID (document ID)
<code>name</code>	string	Full name
<code>email</code>	string	Email address (lowercase)
<code>role</code>	string	<code>Student</code> <code>Faculty</code> <code>Company</code> <code>Admin</code>
<code>status</code>	string	<code>Inactive</code> <code>Active</code> <code>Deactivated</code>
<code>department</code>	string	Academic department
<code>createdAt</code>	timestamp	Registration timestamp

11. Design System

Color Palette

Token	Hex	Usage
<code>primary-500</code>	<code>#4f46e5</code>	Buttons, links, active states (Indigo)
<code>primary-600</code>	<code>#4338ca</code>	Hover states
<code>accent-500</code>	<code>#ec4899</code>	Badges, highlights (Pink)
<code>surface-50</code>	<code>#f8fafc</code>	Page backgrounds
<code>surface-900</code>	<code>#0f172a</code>	Sidebar background

Key CSS Classes (from `index.css`)

Class	Usage
<code>.btn-primary</code>	Primary action button (indigo)
<code>.btn-secondary</code>	Secondary/outline button
<code>.btn-danger</code>	Destructive action (red)
<code>.btn-google</code>	Google SSO button
<code>.input-field</code>	All form inputs
<code>.card</code>	Content cards with shadow
<code>.badge-active</code>	Green status badge
<code>.badge-inactive</code>	Amber status badge
<code>.badge-deactivated</code>	Red status badge
<code>.auth-page</code>	Split-panel auth layout (2 columns)
<code>.sidebar</code>	Dark fixed sidebar
<code>.sidebar-link</code>	Sidebar nav item

12. Testing

Run Tests

```
cd src/backend
npm test
```

Test Coverage

File	Tests
authController	register (valid, duplicate, missing fields), login (correct, wrong pwd, unverified), getProfile
adminController	listUsers, updateRole, updateStatus, deleteUser
Middleware	authenticate, authorize

32/32 tests passing (Jest + Supertest with in-memory Firebase mock)

13. CI/CD

File: .github/workflows/ci.yml

Triggers: Push / PR to main branch

Pipeline Steps: 1. Checkout code 2. Install backend dependencies 3. Run Jest unit tests 4. Report results

14. Sprint Roadmap

Sprint	Name	Status	Key Deliverables
Sprint 1	Setup & Authentication	Complete	Auth system, RBAC, Admin panel, UI design system
Sprint 2	Student Profiles & Job Postings	Planned	Student profile editor, Company job listings, Applications
Sprint 3	Application Tracking	Planned	Application status workflow, notifications

Sprint	Name	Status	Key Deliverables
Sprint 4	Reports & Analytics	Planned	Placement statistics, export to PDF/Excel

Documentation prepared by Team 22MCCE13 & 22MCCE16 — University of Hyderabad, 2026.